Brush

Brush
- 1. Content Description
- 2. Program Startup
- 3. Core Code Analysis

1. Content Description

This lesson implements acquiring the color image and using the MediaPipe framework to detect fingers, then select a color and draw the movement trajectory on the image according to the detected finger movement. This section requires entering commands in the terminal, where the terminal is opened according to the mainboard type. This lesson uses the Raspberry Pi 5 as an example.

For the Raspberry Pi 5 mainboard, you need to open a terminal on the host machine, then enter the command to access the Docker container, and enter the commands mentioned in this lesson inside the terminal after entering the Docker container. For the tutorial on entering the Docker container from the host, please refer to **【Robot Configuration and Operation Guide】, section 【3.Enter Docker (For Raspberry Pi)】** in this product tutorial.

For the Orin and RDK mainboards, simply open a terminal and enter the commands mentioned in this lesson.

2. Program Startup

The Raspberry Pi PI5 controller needs to enter the Docker container first; RDK X5 and Orin controllers do not need to.

To enter the Docker container (for steps, refer to 【Docker Course】 --- 【4. Docker Startup Script】),

All the docker terminals for the commands below need to enter the same Docker container before executing **(for steps, refer to 【Docker Course】 --- 【3.Docker Submission and Multi-Terminal Access】)**

First, enter the following command in the terminal to start the robot camera and low-level driver environment:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

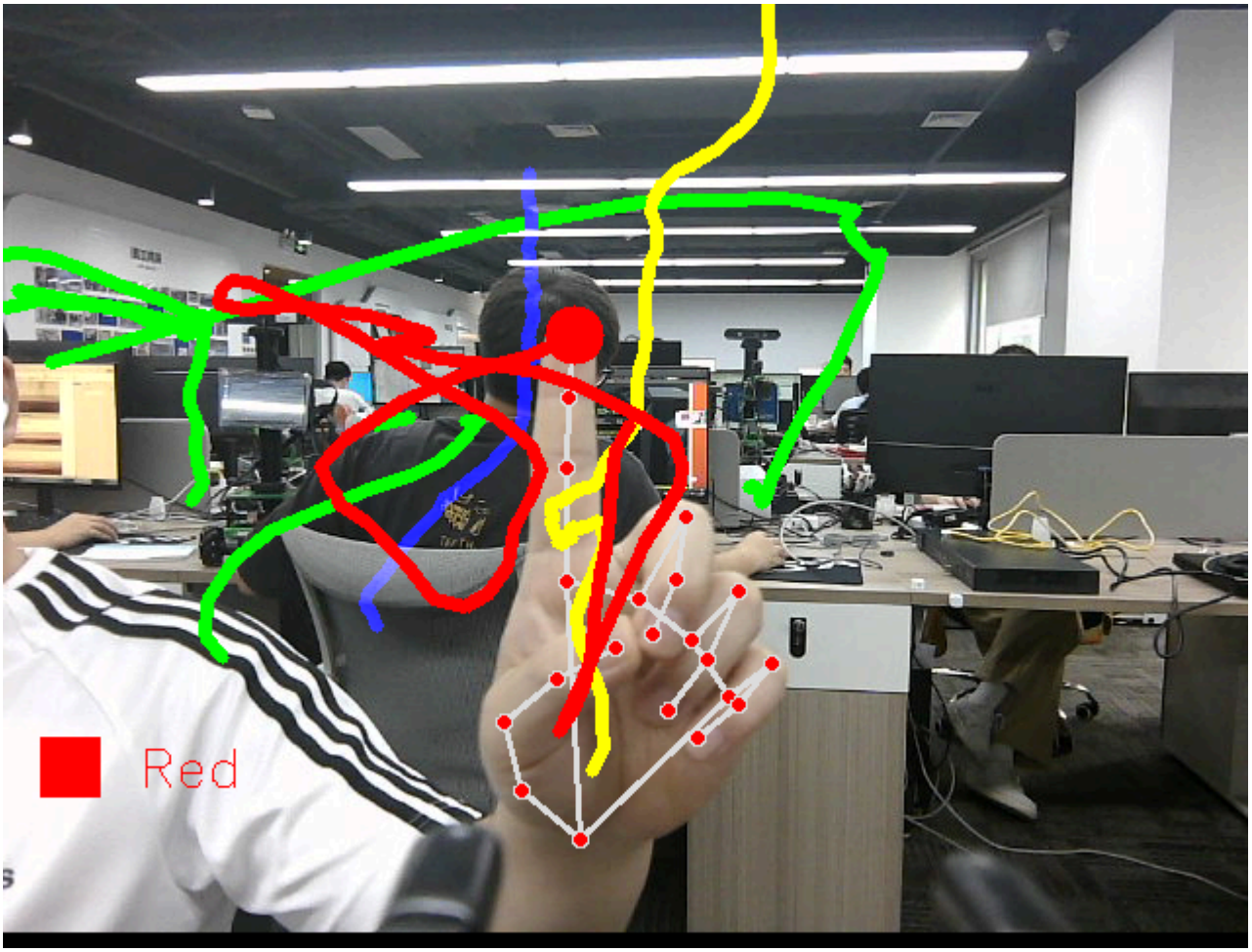
> **Note**
>
> This command is the official all-in-one startup script for the A1Pro, which automatically launches the image stream node according to the camera hardware configuration (USB monocular camera or Nuwa 3D depth camera) and loads the low-level drivers. If you only need to start the camera stream separately, you can also run `ros2 launch usb_cam camera.launch.py` (USB camera) or `ros2 launch ascamera hp60c.launch.py` (Nuwa camera).

After the camera is started successfully, open another terminal and enter the following command in the terminal to start the brush program:

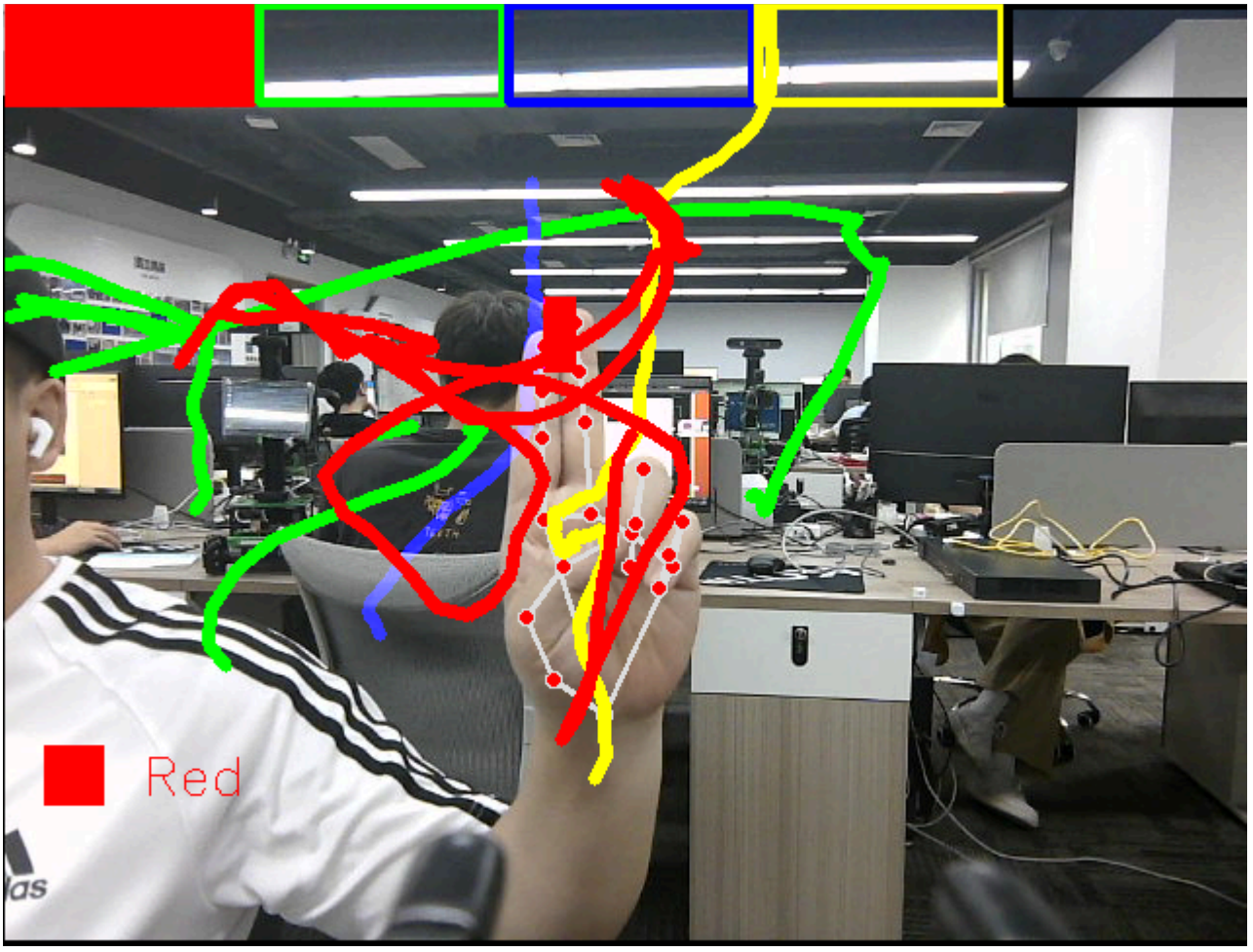
```
ros2 run yahboomcar_mediapipe 08_VirtualPaint
# Control the robotic arm to look forward
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 180,
joint3: 0, joint4: 0, joint5: 90, joint6: 0, time: 1000}" --once
```

After the program runs, as shown in the figure below, the default fingertip color is red. When the index finger and middle finger of the right hand are together, it is in the selection state and a color selection box pops up in the meantime. When the two fingertips move to the corresponding color position, that color is selected (black is the eraser); when the index finger and middle finger are separated, it is in the drawing state, and you can draw freely on the canvas.

When the index finger and middle finger are separated, it is in the drawing state:



When the index finger and middle finger are together, it enters the color selection mode:



3. Core Code Analysis

Program code path:

- Raspberry Pi 5 mainboard

  The program code is located in the running Docker. The path in the Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/08_VirtualPaint.py`

- Orin mainboard

  The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/08_VirtualPaint.py`

- RDK mainboard

  The program code path is `/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/08_VirtualPaint.py`

Import the required library files:

```
import math
import time
import cv2 as cv
import numpy as np
import mediapipe as mp
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import os
```

Initialize the data and define the Publisher and Subscriber:

```
def __init__(self):
    super().__init__('hand_painting_node')
    # Create a blank canvas with a resolution of 480*640
    self.imgCanvas = np.zeros((480, 640, 3), np.uint8)
    # Define the thickness of the brush (red, green, blue, yellow)
    self.brushThickness = 5
    # Define the thickness of the eraser (black)
    self.eraserThickness = 100
    # The y coordinate threshold of the bottom of the color selection box; a value less than
this means entering the color selection mode
    self.top_height = 50
    self.Color = "Red"
    # Define the brush color list; the values after each color name represent the rgb value
of that color
    self.ColorList = {
    'Red': (0, 0, 255),
    'Green': (0, 255, 0),
    'Blue': (255, 0, 0),
    'Yellow': (0, 255, 255),
    'Black': (0, 0, 0),
    }
    # Define the ID of each fingertip, which will be used later to determine which fingers
are straightened
    self.tipIds = [4, 8, 12, 16, 20]
    # Use the class in the mediapipe library to define a hand object
    self.mpHand = mp.solutions.hands
    self.mpDraw = mp.solutions.drawing_utils
    self.hands = self.mpHand.Hands(
    static_image_mode=False,
    max_num_hands=2,
    min_detection_confidence=0.85,
    min_tracking_confidence=0.5 )
    # Define the attributes of the joint connection lines, which will be used in the
subsequent joint connection function
    self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
thickness=-1, circle_radius=15)
    self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255, 0), thickness=10,
circle_radius=10)
```

```
    self.bridge = CvBridge()

    # Define the Subscriber for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type == 'nuwa' else
'/usb_cam/image_raw'

    self.subscription = self.create_subscription(
        Image,
        topic_name,
        self.image_callback,
        10)
```

Color image callback function:

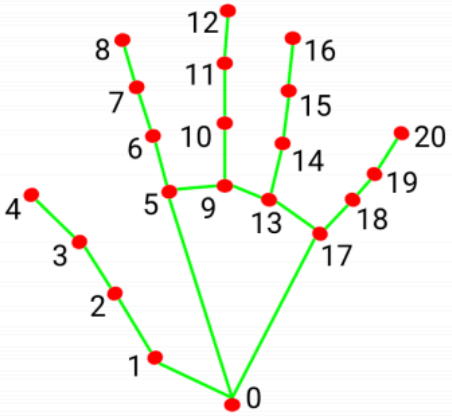
```
def image_callback(self, msg):
    # Convert the color image message data into image data using CvBridge
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    # Obtain the size of the color image for later color selection based on the fingertip
coordinates
    h, w, c = frame.shape
    frame, lmList = self.hand_detector.findHands(frame, draw=False)
    if len(lmList) != 0:
        # print(lmList)
        # tip of index and middle fingers
        # Obtain the fingertip coordinates of the index finger and middle finger
        x1, y1 = lmList[8][1:]
        x2, y2 = lmList[12][1:]
        # Call the fingersUp function to return the list of straightened fingers
        fingers = self.fingersUp()
        # If both the index finger and middle finger are straightened
        if fingers[1] and fingers[2]:
            # print("Seclection mode")
            if y1 < self.top_height:
                # Determine the selected color based on the index finger coordinate and the
size of the image
                if 0 < x1 < int(w / 5) - 1:
                    self.boxx = 0
                    self.Color = "Red"
                if int(w / 5) < x1 < int(w * 2 / 5) - 1:
                    self.boxx = int(w / 5)
                    self.Color = "Green"
                elif int(w * 2 / 5) < x1 < int(w * 3 / 5) - 1:
                    self.boxx = int(w * 2 / 5)
                    self.Color = "Blue"
                elif int(w * 3 / 5) < x1 < int(w * 4 / 5) - 1:
                    self.boxx = int(w * 3 / 5)
                    self.Color = "Yellow"
                elif int(w * 4 / 5) < x1 < w - 1:
                    self.boxx = int(w * 4 / 5)
                    self.Color = "Black"
            cv.rectangle(frame, (x1, y1 - 25), (x2, y2 + 25), self.ColorList[self.Color],
cv.FILLED)
```

```
            cv.rectangle(frame, (self.boxx, 0), (self.boxx + int(w / 5), self.top_height),
self.ColorList[self.Color], cv.FILLED)
            # Draw the color selection box for each color
            cv.rectangle(frame, (0, 0), (int(w / 5) - 1, self.top_height),
self.ColorList['Red'], 3)
            cv.rectangle(frame, (int(w / 5) + 2, 0), (int(w * 2 / 5) - 1, self.top_height),
self.ColorList['Green'], 3)
            cv.rectangle(frame, (int(w * 2 / 5) + 2, 0), (int(w * 3 / 5) - 1,
self.top_height), self.ColorList['Blue'], 3)
            cv.rectangle(frame, (int(w * 3 / 5) + 2, 0), (int(w * 4 / 5) - 1,
self.top_height), self.ColorList['Yellow'], 3)
            cv.rectangle(frame, (int(w * 4 / 5) + 2, 0), (w - 1, self.top_height),
self.ColorList['Black'], 3)
            # If the index finger and middle finger are not straightened simultaneously and
the pixel distance between the two fingertips is greater than 50, it means the fingers are
spread apart, and the program enters the drawing mode at this time
        if fingers[1] and fingers[2] == False and math.hypot(x2 - x1, y2 - y1) > 50:
            # print("Drawing mode")
            if self.xp == self.yp == 0: self.xp, self.yp = x1, y1
            # If the color is black, it is the eraser mode, which erases the drawn
trajectory; otherwise, draw the trajectory on the image according to the selected color
            if self.Color == 'Black':
                cv.line(frame, (self.xp, self.yp), (x1, y1), self.ColorList[self.Color],
self.eraserThickness)
                cv.line(self.imgCanvas, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.eraserThickness)
            else:
                cv.line(frame, (self.xp, self.yp), (x1, y1), self.ColorList[self.Color],
self.brushThickness)
                # Draw the line in the selected color on the created canvas
                cv.line(self.imgCanvas, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.brushThickness)
            cv.circle(frame, (x1, y1), 15, self.ColorList[self.Color], cv.FILLED)
            self.xp, self.yp = x1, y1
        else: self.xp = self.yp = 0
    # Convert the color space of the canvas to a grayscale image for subsequent image
processing
    imgGray = cv.cvtColor(self.imgCanvas, cv.COLOR_BGR2GRAY)
    # Perform thresholding on the grayscale image, that is, divide the pixel values of the
image into two classes (such as foreground and background) according to the set threshold.
It is one of the core methods of image binarization.
    _, imgInv = cv.threshold(imgGray, 50, 255, cv.THRESH_BINARY_INV)
    # Convert the color space and transform the grayscale image into the color BGR image
space
    imgInv = cv.cvtColor(imgInv, cv.COLOR_GRAY2BGR)
    # Perform bitwise AND and OR operations on the images to combine them
    frame = cv.bitwise_and(frame, imgInv)
    frame = cv.bitwise_or(frame, self.imgCanvas)
    # Draw the currently selected color box and color
    cv.rectangle(frame, (20, h - 100), (50, h - 70), self.ColorList[self.Color], cv.FILLED)
    cv.putText(frame, self.Color, (70, h - 75), cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255),
1)
    cv.imshow('frame', frame)
```

The fingersUp function for detecting straightened fingers:

```
def fingersUp(self):
    fingers=[]
    # Thumb, judge whether the angle of each joint of the thumb is greater than 150 degrees; if so, the thumb can be considered straightened
    if (self.calc_angle(self.tipIds[0],
                        self.tipIds[0] - 1,
                        self.tipIds[0] - 2) > 150.0) and (
            self.calc_angle(
                self.tipIds[0] - 1,
                self.tipIds[0] - 2,
                self.tipIds[0] - 3) > 150.0): fingers.append(1)
    else:            fingers.append(0)
    # 4 finger, the remaining four fingers
    for id in range(1, 5):
        # Here it judges whether the y value of the fingertip joint is smaller than that of the middle joint; if so, the finger is considered straightened (the origin of the image is at the upper left, y increases downward, and x increases to the right)
        if self.lmList[self.tipIds[id]][2] < self.lmList[self.tipIds[id] - 2][2]:
            fingers.append(1)
        else:
            fingers.append(0)
    return fingers
```

As shown in the figure below, the ID of each finger joint:



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

The findHands function for palm detection:

```
def findHands(self, frame, draw=True):
    # Create a detection list to store the detection results
    self.lmList = []
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    # Call the process function in the mediapipe library for image processing; during init, the self.hands object was created and initialized
    self.results = self.hands.process(img_RGB)
    if self.results.multi_hand_landmarks:
        for handLms in self.results.multi_hand_landmarks:
```

```
            if draw: self.mpDraw.draw_landmarks(frame, handLms,
self.mpHand.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
            else: self.mpDraw.draw_landmarks(frame, handLms, self.mpHand.HAND_CONNECTIONS)
        # Iterate through the detection results and add the detection results to the
self.lmList list, which represents the id of each detected hand joint and the xy coordinates
of the joint
        for id, lm in enumerate(self.results.multi_hand_landmarks[0].landmark):
            h, w, c = frame.shape
            cx, cy = int(lm.x * w), int(lm.y * h)
            # print(id, cx, cy)
            self.lmList.append([id, cx, cy])
    return frame, self.lmList
```